

Getting Started with

Knockout is a JavaScript library that helps you to create rich, responsive displays and editor UIs with a clean underlying data model. It allows for a graceful sync between the UI and the model.

Let us first look at the main features of Knockout.js, which include:

- Dependency tracking
- The flexibility to update the appropriate sections in the UI as and when required
- Declarative binding
- Extensibility

How is Knockout different?

Knockout doesn't compete with jQuery or other such similar low-level DOM APIs. It provides a complementary, high-level way to link a data model to a UI. Knockout (KO) itself doesn't depend on jQuery, but you can certainly use jQuery at the same time.

Knockout uses the MVVM design pattern.

Model-View-View Model (MVVM) is a design pattern for building user interfaces. It allows you to keep a potentially sophisticated UI simple by splitting it into three parts.

- **Model:** This is your application's stored data. This data represents objects and operations in your business domain (e.g., the employee information of a company) and is independent of any UI.
- **View model:** This is a pure code representation of the data and operations on a UI. For example, if you're implementing a list editor, your view model would be an object holding a list of items, and exposing methods to add and remove items. Do note that this is not the UI itself — it doesn't have any concept of

buttons or display styles. It's not the persisted data model either — it holds the unsaved data the user is working with. When using KO, view models are pure JavaScript objects that hold no knowledge of HTML. Keeping the view model abstract in this way lets it stay simple, so you can manage more sophisticated behaviour without getting lost.

- **View:** This is a visible, interactive UI representing the state of the view model. It displays information from the view model, sends commands to it (e.g., when the user clicks on a button), and updates whenever the state of the view model changes.

When using KO, your view is simply an HTML document with declarative bindings to link it to the view model. Alternatively, you can use templates that generate HTML, using data from your view model.

To create a view model with KO, declare any JavaScript object. For example:

```
var employeeModel={
  name:'Anna',
  department:'Sales'
};
```

You can then create a very simple view of this view model using a declarative binding. For example, the following markup displays the department value:

```
<p>The employee belongs to the
department <span data-bind="text:
```

```
department"></span></p>
```

Downloading and installing Knockout

Get the latest version of Knockout.js from <https://knockoutjs.com/downloads> and reference it using a `<script>` tag somewhere on your HTML pages. For example:

```
<script type='text/javascript'
src='knockout-3.5.1.js'></script>
```

Update the `src` attribute to match the location where you put the downloaded file.

Activating Knockout

The `data-bind` attribute isn't native to HTML, though it is perfectly okay (it's strictly compliant in HTML 5 and causes no problems with HTML 4 even though a validator will point out that it's an unrecognised attribute). But since the browser doesn't know what it means, you need to activate Knockout to make it take effect.

To activate Knockout, add the following line to a `<script>` block:

```
ko.applyBindings(myViewModel);
```

You can either put the script block at the bottom of the HTML document, or you can put it at the top and wrap the contents in a DOM-ready handler such as jQuery's `$` function.

Now the view will be displayed as if you'd written the following HTML:

```
The name is <span>Bob</span>
```